Guía 1 de 3 — APIs avanzadas: paginación y combinar dos fuentes

DPL1046 · Clase 8 · Para repasar en casa, a tu ritmo.

En proyectos reales los datos vienen **por partes** y desde **más de un lugar**. Esta guía repasa cómo pedir una API **página por página** (`while`) y cómo **combinar un CSV con una API** en una sola tabla.

> Antes de empezar, haz tu rutina de inicio (Guía 1 de la Clase 2) con el `.venv` activado. Se asume que ya repasaste la Guía 2 de la Clase 4 (APIs, `requests` y `try / except`).

---

1. Paginación: los datos llegan por partes

Piensa en un **buscador** que muestra los resultados de 10 en 10 y abajo dice "Siguiente". Una API grande hace lo mismo: no manda un millón de registros de golpe, los entrega por **páginas**. Cada respuesta trae **un trozo de datos y la dirección de la página siguiente**. Hay que seguir pidiendo hasta que no haya más.

El `while`: repetir mientras algo sea verdad

El `for` repite *para cada elemento* de una lista. El `while` repite **mientras se cumpla una condición**, aunque no sepas cuántas veces. Es **llenar una botella mientras no esté llena**: no cuentas los chorros, miras la botella.

> **Cuidado:** si la condición nunca deja de cumplirse, el programa **no termina**. Por eso ponemos un tope (`MAX_PAGINAS`).

```
import requests
import pandas as pd

url = "https://pokeapi.co/api/v2/pokemon?limit=20"   # 20 registros por página
MAX_PAGINAS = 3                                       # tope para la clase

registros = []
pagina = 0

try:
    while url and pagina < MAX_PAGINAS:
        pagina = pagina + 1
        respuesta = requests.get(url, timeout=10)
        respuesta.raise_for_status()
        datos = respuesta.json()
        registros.extend(datos["results"])
        print("Página", pagina, "→ total acumulado:", len(registros))
        url = datos["next"]

    tabla = pd.DataFrame(registros)
    print(tabla.head(3))
    print("Filas en la tabla final:", len(tabla))
except requests.exceptions.RequestException:
    print("No pude conectar con la API. Revisa tu internet.")
```

Línea por línea:

- `url = "...?limit=20"` → la primera página. `?limit=20` pide 20 registros por página. (PokeAPI es pública y sin clave; el patrón es el mismo en cualquier API.)
- `MAX_PAGINAS = 3` → una **constante** (se escribe en mayúsculas): un valor que no cambia.
- `registros = []` y `pagina = 0` → una lista vacía para acumular y un contador.
- `while url and pagina < MAX_PAGINAS:` → "mientras exista una URL **y** no hayamos llegado al tope". `url` es verdadera solo si tiene texto; cuando la API no tenga siguiente página será `None` y el ciclo termina.
- `registros.extend(datos["results"])` → suma los registros de esta página **uno por uno** a la lista grande. (Con `append` agregarías la lista entera como *un solo* elemento.)
- `url = datos["next"]` → **la línea clave**: la respuesta trae la dirección de la página siguiente; la guardamos para que el `while` pida la próxima.
- `pd.DataFrame(registros)` → lista de diccionarios → tabla (60 filas: 3 páginas × 20).

Palabra	Qué hace
`limit`	Cuántos registros por página
`next`	Dirección de la página siguiente (`None` si no hay más)
`while`	Repite mientras la condición sea verdadera
`MAX_PAGINAS`	Tope de seguridad

---

2. Combinar dos fuentes en una sola tabla

Quiero saber si las ventas se mueven con el dólar. Las **ventas** están en un CSV y el **dólar** en una API. Cada fuente es una pieza de un puzzle; las junto con una **llave en común**: el **mes**.

Fuente 1: ventas por mes (CSV)

```
orders = pd.read_csv(RUTA + "olist_orders_dataset.csv")
items = pd.read_csv(RUTA + "olist_order_items_dataset.csv")

tabla = items.merge(orders, on="order_id", how="inner")
tabla["mes"] = pd.to_datetime(tabla["order_purchase_timestamp"]).dt.strftime("%Y-
%m")

ventas = tabla.groupby("mes")["price"].sum().round(0).reset_index()
ventas = ventas[ventas["mes"].str.startswith("2017")]
```

- `dt.strftime("%Y-%m")` **formatea** la fecha como texto “año-mes”, por ejemplo `2017-10`. Ese texto será la llave.
- `reset_index()` hace que `mes` vuelva a ser una **columna normal** (si no, queda como etiqueta de fila y no se puede usar en el `merge`).
- `.str.startswith("2017")` aplica “empieza con” a cada texto: nos quedamos con los meses de 2017.

Fuente 2: el dólar (API)

```
try:
    respuesta = requests.get("https://mindicador.cl/api/dolar/2017", timeout=15)
    respuesta.raise_for_status()
    serie = respuesta.json()["serie"]                # lista de {"fecha",
"valor"}

    dolar = pd.DataFrame(serie)
    dolar["mes"] = pd.to_datetime(dolar["fecha"]).dt.strftime("%Y-%m")
    dolar_mes = dolar.groupby("mes")["valor"].mean().round(1).reset_index()
    dolar_mes = dolar_mes.rename(columns={"valor": "dolar_clp"})

    combinada = ventas.merge(dolar_mes, on="mes", how="left")
    print(combinada)
except requests.exceptions.RequestException:
    print("No pude conectar con la API. Las ventas sí se calcularon:")
    print(ventas)
```

- El dólar es **diario** y las ventas **mensuales**: promediamos el dólar de cada mes con `groupby("mes")["valor"].mean()`.
- Creamos el texto "año-mes" **con el mismo formato** en ambas tablas: deben hablar el mismo idioma para poder unirse.
- `rename(columns={"valor": "dolar_clp"})` renombra la columna a algo con significado.
- `merge(..., on="mes", how="left")` → una fila por mes con las dos fuentes. Con `left` **conservamos todos los meses de ventas**, tengan o no dólar.
- El `except` hace que, sin internet, el programa **no se caiga**: muestra igual las ventas, que vienen de archivos locales. Un pipeline que falla con elegancia.

---

3. Buenas prácticas con APIs

- Siempre `timeout` y `try / except`.
- Pon un **tope** a los ciclos `while`.
- Sé amable con el servicio: no lo bombardees con cientos de peticiones seguidas.
- Guarda lo que bajaste (CSV o Parquet) para no pedirlo de nuevo en cada prueba.
- Las claves van en `.env` (Guía 3 de la Clase 5).

Errores frecuentes

Qué ves	Por qué pasa	Qué hacer
El script no termina	El `while` nunca deja de cumplirse	Agrega un tope y revisa que `url` cambie en cada vuelta
Siempre trae solo la primera página	Olvidaste `url = datos["next"]`	Actualiza la `url` dentro del ciclo
El `merge` deja `NaN` en el dólar	Los formatos de `mes` no coinciden	Imprime ambos `mes` y compáralos
`KeyError: 'mes'` en el `merge`	Falta `reset_index()`	Agrégalo después del `groupby`
`timeout`	Internet lento o API caída	Aumenta el `timeout` o intenta más tarde; el `except` debe avisar

Practica tú

1. ¿Qué cambias para traer **5** páginas en vez de 3?
2. ¿Cuántas filas debería tener la tabla final con 5 páginas de 20?
3. ¿Por qué se usa `how="left"` y no `inner` al unir ventas con el dólar?
4. ¿Qué haría `ventas[ventas["mes"].str.startswith("2018")]`?

▼ **Ver respuestas**

1. `MAX_PAGINAS = 5`.
2. 100 filas (5 × 20).
3. Para no perder meses de ventas si falta el dólar de ese mes.
4. Se queda solo con los meses de 2018.

Para llevarte

- Una API grande entrega **por páginas**; cada respuesta trae la **siguiente** en `next`.
- `while` repite mientras algo sea verdad: **siempre con un tope**.
- Para combinar fuentes: **misma llave, mismo formato**, y `merge`.
- Un pipeline robusto avisa si una fuente falla, sin caerse.